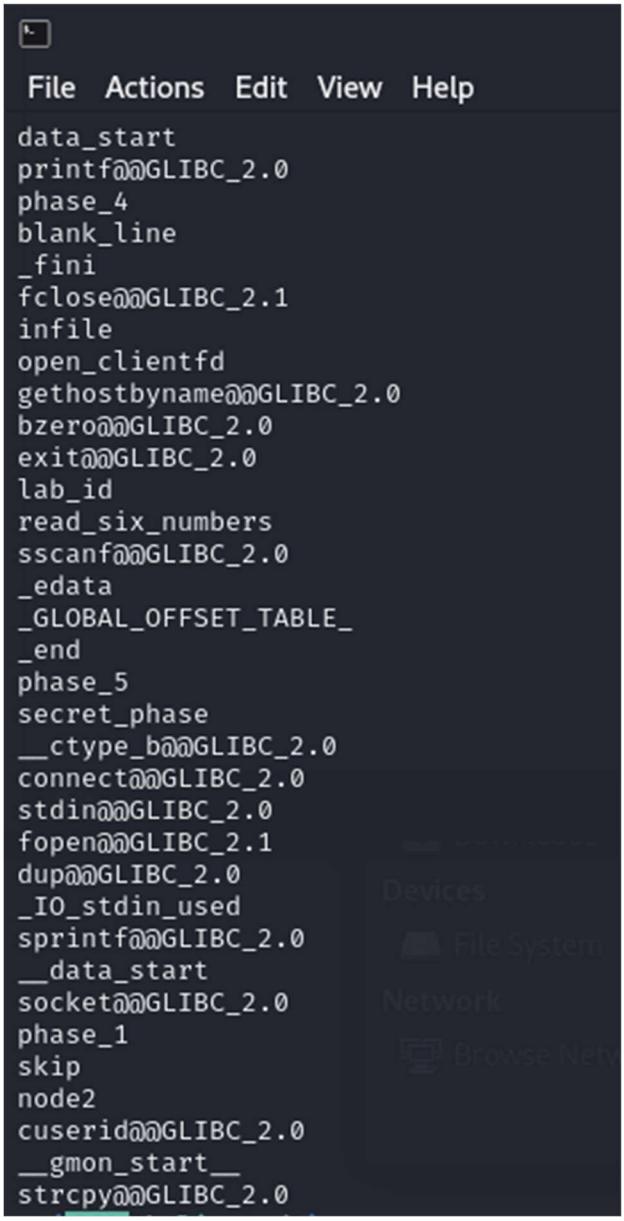
James Stallkamp

CS373

Lab 1

I started off lab 1 using the strings command to find important pieces of the syntax and immediately saw things like phase_1 through phase_6, as well as a secret and much more useful information to defuse this bomb.

Strings bomb:

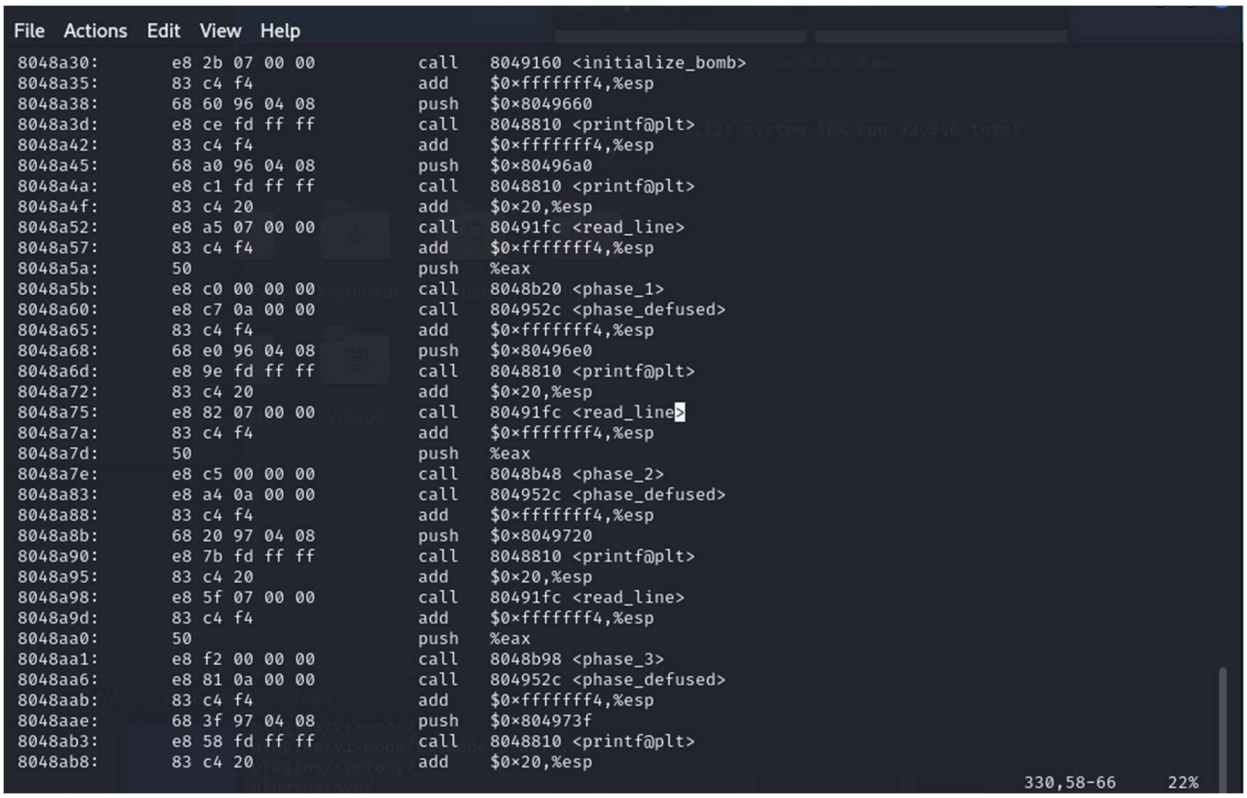


Phase 1

I then examined the object dump file for this file and quickly found sections in the assembly for each phase of the bomb.

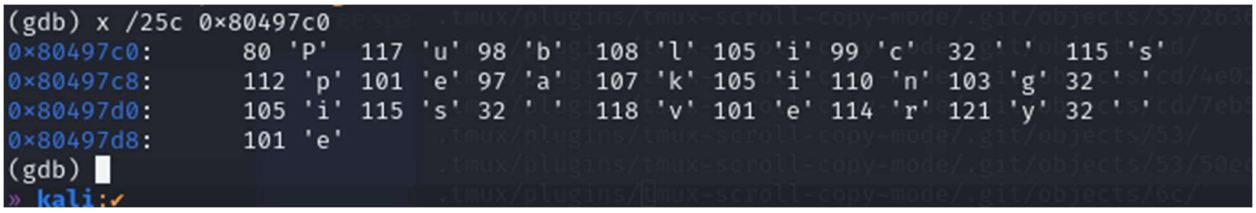
Used objdump -d bomb > bomb-assembly

Looked at assembly code found



I found that for phase 1 it was comparing a string stored in eax so I decided to examine what charaters were in that register

Checking eax



I found characters “public speaking is very e”

I checked strings and found “public speaking is very easy.” That is probably password 1.

Phase 1 password : public speaking is very easy.

```
┌─(ROOT@kali:pts/3)──────────────────────────────────────────────(~/Labs/Lab1)─┐
└─(8:00:47:#)── ./bomb                                            ──(Mon,Oct17)─┘
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Public speaking is very easy.
Phase 1 defused. How about the next one?
```

Phase 2

After successfully doing phase 1 I went back to object dump file and looked at phase 2 and found the read_six_numbers being called.

```
 8048b59:       50                      push   %eax
 8048b5a:       52                      push   %edx
 8048b5b:       e8 78 04 00 00          call   8048fd8 <read_six_numbers>
 8048b60:       83 c4 10                add    $0x10,%esp
 8048b63:       83 7d e8 01             cmpl   $0x1,-0x18(%ebp)
 8048b67:       74 05                   je     8048b6e <phase_2+0x26>
 8048b69:       e8 8e 09 00 00          call   80494fc <explode_bomb>
 8048b6e:       bb 01 00 00 00          mov    $0x1,%ebx
 8048b73:       8d 75 e8                lea    -0x18(%ebp),%esi
 8048b76:       8d 43 01                lea    0x1(%ebx),%eax
 8048b79:       0f af 44 9e fc          imul   -0x4(%esi,%ebx,4),%eax
```

I checked Func read_six_numbers and found this. This is reading in 6 numbers in the format “1 2 3 4 5 6”

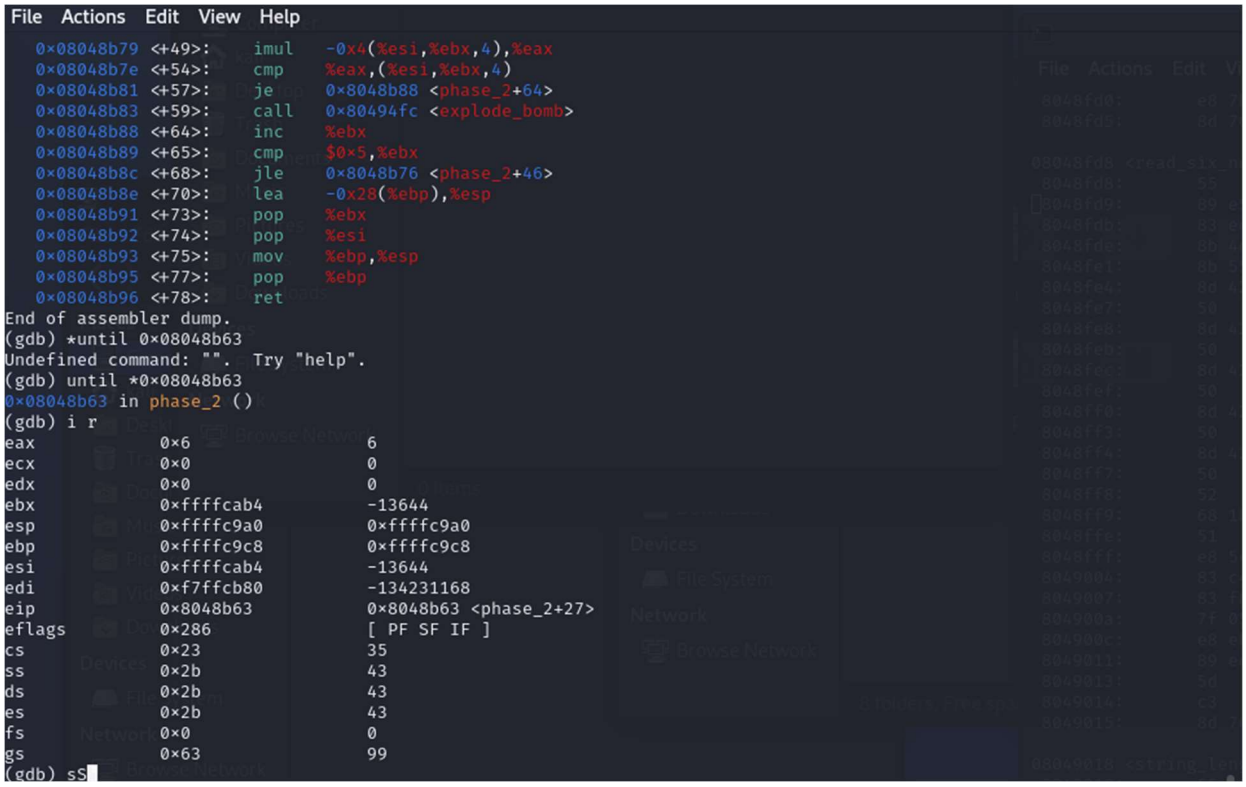
```
08048fd8 <read_six_numbers>:
 8048fd8:       55                      push   %ebp
 8048fd9:       89 e5                   mov    %esp,%ebp
 8048fdb:       83 ec 08                sub    $0x8,%esp
 8048fde:       8b 4d 08                mov    0x8(%ebp),%ecx
 8048fe1:       8b 55 0c                mov    0xc(%ebp),%edx
 8048fe4:       8d 42 14                lea    0x14(%edx),%eax
 8048fe7:       50                      push   %eax
 8048fe8:       8d 42 10                lea    0x10(%edx),%eax
 8048feb:       50                      push   %eax
 8048fec:       8d 42 0c                lea    0xc(%edx),%eax
 8048fef:       50                      push   %eax
 8048ff0:       8d 42 08                lea    0x8(%edx),%eax
 8048ff3:       50                      push   %eax
 8048ff4:       8d 42 04                lea    0x4(%edx),%eax
 8048ff7:       50                      push   %eax
 8048ff8:       52                      push   %edx
 8048ff9:       68 1b 9b 04 08          push   $0x8049b1b
 8048ffe:       51                      push   %ecx
 8048fff:       e8 5c f8 ff ff          call   8048860 <sscanf@plt>
 8049004:       83 c4 20                add    $0x20,%esp
 8049007:       83 f8 05                cmp    $0x5,%eax
 804900a:       7f 05                   jg     8049011 <read_six_numbers+0x39>
 804900c:       e8 eb 04 00 00          call   80494fc <explode_bomb>
 8049011:       89 ec                   mov    %ebp,%esp
 8049013:       5d                      pop    %ebp
 8049014:       c3                      ret
 8049015:       8d 76 00                lea    0x0(%esi),%esi
```

So using gdb I decided to debug what happens when I try six random numbers like 1 2 3 4 5 6.

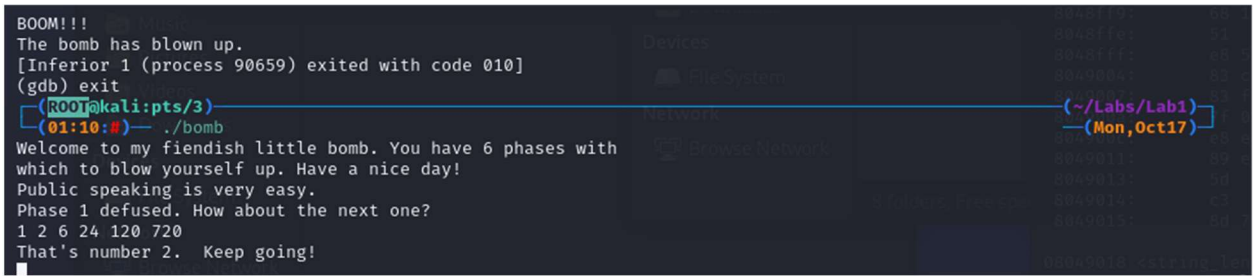
```
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
    <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from bomb...
(gdb) break phase_2
Breakpoint 1 at 0x8048b50
(gdb) run
Starting program: /root/Labs/Lab1/bomb
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Public speaking is very easy.
Phase 1 defused. How about the next one?
1 2 3 4 5 6
```

```
Breakpoint 1, 0x08048b50 in phase_2 ()
(gdb) disas
Dump of assembler code for function phase_2:
   0x08048b48 <+0>:     push   %ebp
   0x08048b49 <+1>:     mov    %esp,%ebp
   0x08048b4b <+3>:     sub    $0x20,%esp
   0x08048b4e <+6>:     push   %esi
   0x08048b4f <+7>:     push   %ebx
=> 0x08048b50 <+8>:     mov    0x8(%ebp),%edx
   0x08048b53 <+11>:    add    $0xfffffff8,%esp
   0x08048b56 <+14>:    lea    -0x18(%ebp),%eax
   0x08048b59 <+17>:    push   %eax
   0x08048b5a <+18>:    push   %edx
   0x08048b5b <+19>:    call   0x8048fd8 <read_six_numbers>
   0x08048b60 <+24>:    add    $0x10,%esp
   0x08048b63 <+27>:    cmpl   $0x1,-0x18(%ebp)
   0x08048b67 <+31>:    je     0x8048b6e <phase_2+38>
   0x08048b69 <+33>:    call   0x80494fc <explode_bomb>
   0x08048b6e <+38>:    mov    $0x1,%ebx
   0x08048b73 <+43>:    lea    -0x18(%ebp),%esi
   0x08048b76 <+46>:    lea    0x1(%ebx),%eax
   0x08048b79 <+49>:    imul   -0x4(%esi,%ebx,4),%eax
   0x08048b7e <+54>:    cmp    %eax,(%esi,%ebx,4)
   0x08048b81 <+57>:    je     0x8048b88 <phase_2+64>
   0x08048b83 <+59>:    call   0x80494fc <explode_bomb>
   0x08048b88 <+64>:    inc    %ebx
   0x08048b89 <+65>:    cmp    $0x5,%ebx
   0x08048b8c <+68>:    jle    0x8048b76 <phase_2+46>
   0x08048b8e <+70>:    lea    -0x28(%ebp),%esp
   0x08048b91 <+73>:    pop    %ebx
   0x08048b92 <+74>:    pop    %esi
   0x08048b93 <+75>:    mov    %ebp,%esp
   0x08048b95 <+77>:    pop    %ebp
   0x08048b96 <+78>:    ret
End of assembler dump.
```

I found at 0x08048b63 it is comparing against 1 so first num must be 1! I then found a loop where it goes up to 5 and it probably comparing the rest of the numbers.



Use I r command to see contents of registers that was being compared, went through the loop and found 1 2 6 24 120 720. Which must be the password for the second phase



Phase 2 password: 1 2 6 24 120 720

Phase 3

Going back to the object dump file I went to the lines for phase 3 and noticed it was using scanf and then comparing several values in a certain location. So I decided to use a test string and look at what those values are.

```
8048bb6:       52                      push   %edx
8048bb7:       e8 a4 fc ff ff          call   8048860 <sscanf@plt>
8048bbc:       83 c4 20                add    $0×20,%esp
8048bbf:       83 f8 02                cmp    $0×2,%eax
8048bc2:       7f 05                   jg     8048bc9 <phase_3+0×31>
8048bc4:       e8 33 09 00 00          call   80494fc <explode_bomb>
8048bc9:       83 7d f4 07             cmpl   $0×7,-0×c(%ebp)
8048bcd:       0f 87 b5 00 00 00       ja     8048c88 <phase_3+0×f0>
8048bd3:       8b 45 f4                mov    -0×c(%ebp),%eax
8048bd6:       ff 24 85 e8 97 04 08    jmp    *0×80497e8(,%eax,4)
8048bdd:       8d 76 00                lea    0×0(%esi),%esi
8048be0:       b3 71                   mov    $0×71,%bl
8048be2:       81 7d fc 09 03 00 00    cmpl   $0×309,-0×4(%ebp)
8048be9:       0f 84 a0 00 00 00       je     8048c8f <phase_3+0×f7>
8048bef:       e8 08 09 00 00          call   80494fc <explode_bomb>
8048bf4:       e9 96 00 00 00          jmp    8048c8f <phase_3+0×f7>
8048bf9:       8d b4 26 00 00 00 00    lea    0×0(%esi,%eiz,1),%esi
8048c00:       b3 62                   mov    $0×62,%bl
```

```
Breakpoint 1, 0×08048b9f in phase_3 ()
(gdb) x/s 0×0804b721
0×804b721 <input_strings+161>:  " 1 2"
(gdb) x/s 0×080497de
0×80497de:      "%d %c %d"
(gdb) p/x $edx
$1 = 0×804b721
(gdb) x/s 0×0804b721
0×804b721 <input_strings+161>:  " 1 2"
(gdb)
```

Here I was able to see that the format of the next key must be “int char int” and that the first number is 1 and followed by a 2, which seems odd since the format should have been int char int. This makes me think maybe 2 is actually part of the second number and the character is being checked elsewhere. I found that it was then comparing against the value $0xd6 which corresponds to value 214 which when combined with 2 means that the second integer must be 214.

For the character I found it was comparing to the value in -0x5(%ebp).

```
 8048c88:       b3 78                   mov    $0×78,%bl
 8048c8a:       e8 6d 08 00 00          call   80494fc <explode_bomb>
 8048c8f:       3a 5d fb                cmp    -0×5(%ebp),%bl
 8048c92:       74 05                   je     8048c99 <phase_3+0×101>
 8048c94:       e8 63 08 00 00          call   80494fc <explode_bomb>
 8048c99:       8b 5d e8                mov    -0×18(%ebp),%ebx
```

I exmained that register and found 62 in hex which corresponds to 98 in decimal and the character ‘b’.

So the password must be 1 b 214

```
./bomb
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Public speaking is very easy.
Phase 1 defused. How about the next one?
1 2 6 24 120 720
That's number 2.  Keep going!
1 b 214
Halfway there!
```

Phase 4

Looking at the assembly for phase 4 I found this func4 function that was being used. I also immediately recognized that func4 was calling itself so we are dealing with a recursive function.

```
08048ca0 <func4>:
 8048ca0:       55                      push   %ebp
 8048ca1:       89 e5                   mov    %esp,%ebp
 8048ca3:       83 ec 10                sub    $0x10,%esp
 8048ca6:       56                      push   %esi
 8048ca7:       53                      push   %ebx
 8048ca8:       8b 5d 08                mov    0x8(%ebp),%ebx
 8048cab:       83 fb 01                cmp    $0x1,%ebx
 8048cae:       7e 20                   jle    8048cd0 <func4+0x30>
 8048cb0:       83 c4 f4                add    $0xfffffff4,%esp
 8048cb3:       8d 43 ff                lea    -0x1(%ebx),%eax
 8048cb6:       50                      push   %eax
 8048cb7:       e8 e4 ff ff ff          call   8048ca0 <func4>
 8048cbc:       89 c6                   mov    %eax,%esi
 8048cbe:       83 c4 f4                add    $0xfffffff4,%esp
 8048cc1:       8d 43 fe                lea    -0x2(%ebx),%eax
 8048cc4:       50                      push   %eax
 8048cc5:       e8 d6 ff ff ff          call   8048ca0 <func4>
 8048cca:       01 f0                   add    %esi,%eax
 8048ccc:       eb 07                   jmp    8048cd5 <func4+0x35>
 8048cce:       89 f6                   mov    %esi,%esi
 8048cd0:       b8 01 00 00 00          mov    $0x1,%eax
 8048cd5:       8d 65 e8                lea    -0x18(%ebp),%esp
 8048cd8:       5b                      pop    %ebx
 8048cd9:       5e                      pop    %esi
 8048cda:       89 ec                   mov    %ebp,%esp@sS
 8048cdc:       5d                      pop    %ebp
 8048cdd:       c3                      ret
 8048cde:       89 f6                   mov    %esi,%esi
```

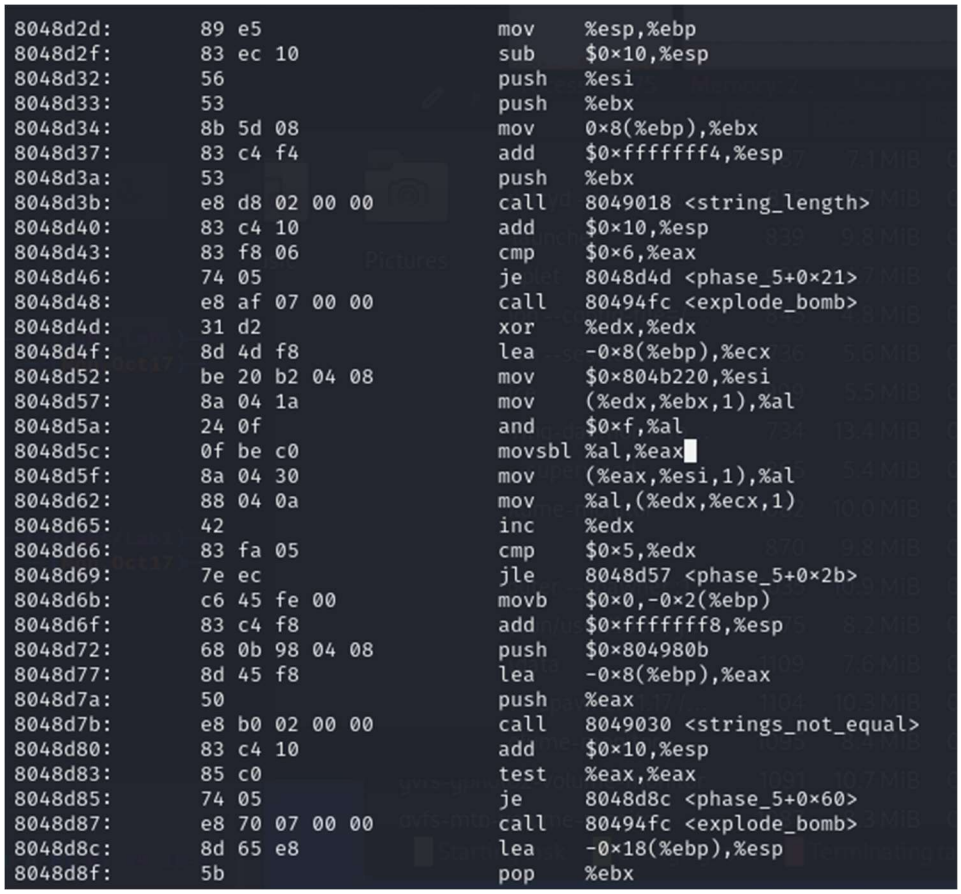
To solve this recursive function my first thought was to find the base case, which I found because the function was campaign its input to 1, so this must be the base case. During the recursive step the

function calls itself twice and passes input-1 and input-2 into the calls. So it appears that f(x) = F(x-1)+f(x-2).

Going back to the assembly for phase 4 I found that whatever comes out of func4 has to equal hex 37 or dec 55. The function from func4 is clearly a Fibonacci sequence so we need a fib number to generate 55 and remember it will need to be n-1. So Fib number 10 corresponds to 55, meaning the password should be 9.

Phase 5

Staring on phase 5, looking at object code.



```
08049018 <string_length>:
 8049018:       55                      push   %ebp
 8049019:       89 e5                   mov    %esp,%ebp
 804901b:       8b 55 08                mov    0x8(%ebp),%edx
 804901e:       31 c0                   xor    %eax,%eax
 8049020:       80 3a 00                cmpb   $0x0,(%edx)
 8049023:       74 07                   je     804902c <string_length+0x14>
 8049025:       42                      inc    %edx
 8049026:       40                      inc    %eax
 8049027:       80 3a 00                cmpb   $0x0,(%edx)
 804902a:       75 f9                   jne    8049025 <string_length+0xd>
 804902c:       89 ec                   mov    %ebp,%esp
 804902e:       5d                      pop    %ebp
 804902f:       c3                      ret
```

Looking at the assembly for phase 5 I quickly noticed several things. I found a string_length function and a register being used in a loop. I found the value being compared to be 6 which means the password for this phase must by 6 long. I also noticed that it was comparing against characters, so we are looking for 6 characters. I found it was comparing against a string located at 0x804980b which I decided to investigate while doing a test run.

```
(gdb) x/6c 0x804980b
0x804980b:      103 'g' 105 'i' 97 'a'  110 'n' 116 't' 115 's'
(gdb)
» kali:✔
```

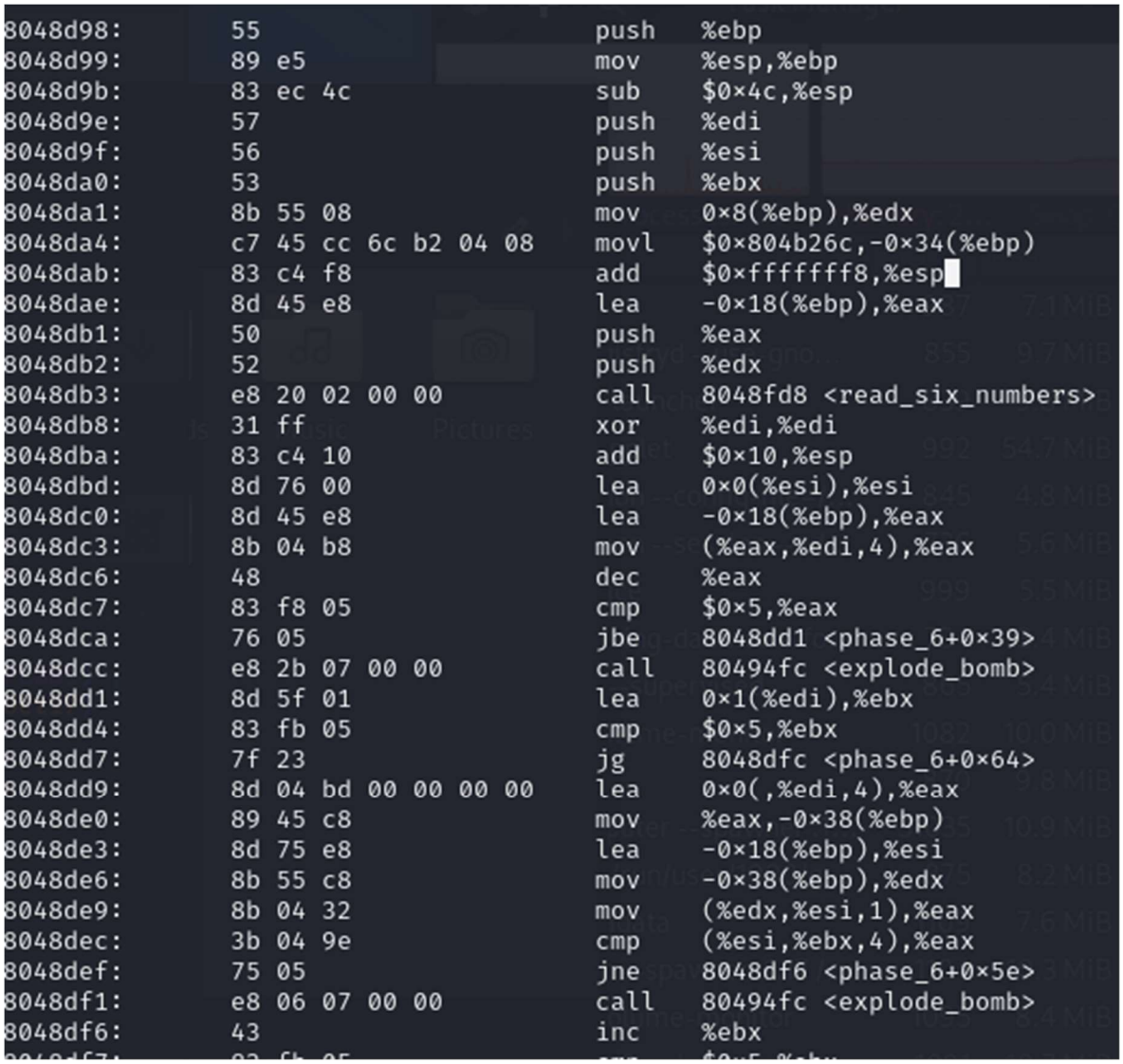
I found the string “giants” in the location and when I checked the compare I found that my input did not match the string that I entered. So the function here must be running some kind of cipher on my input and then comparing against the string “giants”

What I did notice was that the cipher seemed consistent and was therefore simply remapping letters. So I did some brute force mapping characters and found that “opekmq” mapped to giants’

```
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Public speaking is very easy.
Phase 1 defused. How about the next one?
1 2 6 24 120 720
That's number 2.  Keep going!
1 b 214
Halfway there!
9
So you got that one.  Try this one.
opekmq
Good work!  On to the next...
» kali:✔                                                    0:zsh* root:A
```

Password: opekmq

Phase 6



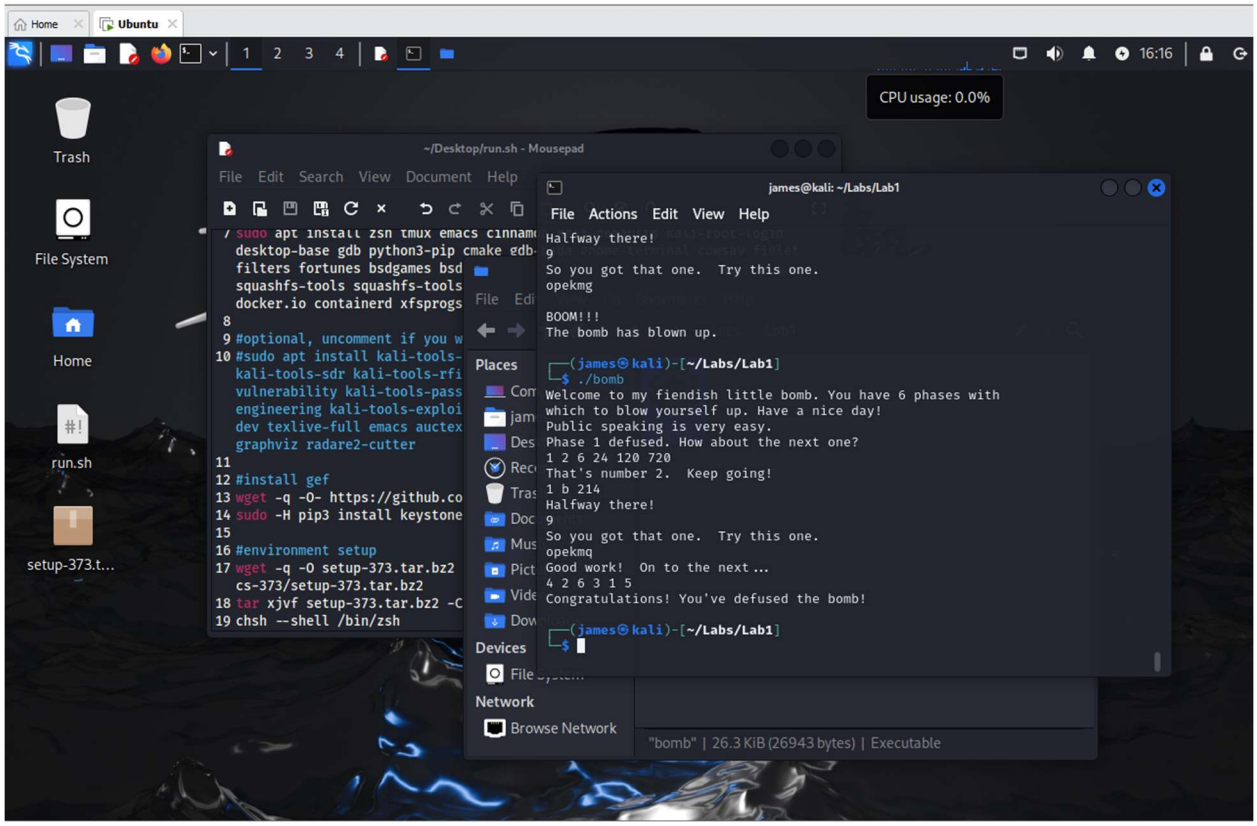
```
8048d98:    55                      push   %ebp
8048d99:    89 e5                   mov    %esp,%ebp
8048d9b:    83 ec 4c                sub    $0×4c,%esp
8048d9e:    57                      push   %edi
8048d9f:    56                      push   %esi
8048da0:    53                      push   %ebx
8048da1:    8b 55 08                mov    0×8(%ebp),%edx
8048da4:    c7 45 cc 6c b2 04 08    movl   $0×804b26c,-0×34(%ebp)
8048dab:    83 c4 f8                add    $0×fffffff8,%esp
8048dae:    8d 45 e8                lea    -0×18(%ebp),%eax
8048db1:    50                      push   %eax
8048db2:    52                      push   %edx
8048db3:    e8 20 02 00 00          call   8048fd8 <read_six_numbers>
8048db8:    31 ff                   xor    %edi,%edi
8048dba:    83 c4 10                add    $0×10,%esp
8048dbd:    8d 76 00                lea    0×0(%esi),%esi
8048dc0:    8d 45 e8                lea    -0×18(%ebp),%eax
8048dc3:    8b 04 b8                mov    (%eax,%edi,4),%eax
8048dc6:    48                      dec    %eax
8048dc7:    83 f8 05                cmp    $0×5,%eax
8048dca:    76 05                   jbe    8048dd1 <phase_6+0×39>
8048dcc:    e8 2b 07 00 00          call   80494fc <explode_bomb>
8048dd1:    8d 5f 01                lea    0×1(%edi),%ebx
8048dd4:    83 fb 05                cmp    $0×5,%ebx
8048dd7:    7f 23                   jg     8048dfc <phase_6+0×64>
8048dd9:    8d 04 bd 00 00 00 00    lea    0×0(,%edi,4),%eax
8048de0:    89 45 c8                mov    %eax,-0×38(%ebp)
8048de3:    8d 75 e8                lea    -0×18(%ebp),%esi
8048de6:    8b 55 c8                mov    -0×38(%ebp),%edx
8048de9:    8b 04 32                mov    (%edx,%esi,1),%eax
8048dec:    3b 04 9e                cmp    (%esi,%ebx,4),%eax
8048def:    75 05                   jne    8048df6 <phase_6+0×5e>
8048df1:    e8 06 07 00 00          call   80494fc <explode_bomb>
8048df6:    43                      inc    %ebx
```

I started looking at the code for phase_6 and noticed that read_six_numbers is back so we are looking for 6 spaced out numbers again. I also noticed a compare that was checking if each number was larger than 5 and a bunch of loops that I didn't understand. So far I know the password for this phase is 6 integers spaced out and they have to be less than 6, so I decided to go in and test.

I used the test string 1 2 3 4 5 6 and looked at the contents of %esi and %esi + 8. From This it appears that we are working with a linked list because each node has an element and a pointer to the next node. Based on the compare that I noticed earlier I tried putting the nodes in order from largest to smallest.

The values were node 1: 0x000000fd 2: 0x000002d5 3: 0x0000012d 4: 0x000003e5 5: 0x000000d4 6: 0x000001b0.

Putting those in order we get 4 2 6 3 1 5


Home
Ubuntu
1 2 3 4
16:16
CPU usage: 0.0%
Trash
File System
Home
run.sh
setup-373.t...
~/Desktop/run.sh - Mousepad
File Edit Search View Document Help
7 sudo apt install zsh tmux emacs cinnam
desktop-base gdb python3-pip cmake gdb-
filters fortunes bsdgames bsd
squashfs-tools squashfs-tools
docker.io containerd xfsprogs
8
9 #optional, uncomment if you w
10 #sudo apt install kali-tools-
kali-tools-sdr kali-tools-rfi
vulnerability kali-tools-pass
engineering kali-tools-exploi
dev texlive-full emacs auctex
graphviz radare2-cutter
11
12 #install gef
13 wget -q -O- https://github.co
14 sudo -H pip3 install keystone
15
16 #environment setup
17 wget -q -O setup-373.tar.bz2
cs-373/setup-373.tar.bz2
18 tar xjvf setup-373.tar.bz2 -C
19 chsh --shell /bin/zsh
Places
Devices
Network
Browse Network
james@kali: ~/Labs/Lab1
File Actions Edit View Help
Halfway there!
9
So you got that one. Try this one.
opekmg
BOOM!!!
The bomb has blown up.
(james㉿kali)-[~/Labs/Lab1]
$ ./bomb
Welcome to my fiendish little bomb. You have 6 phases with
which to blow yourself up. Have a nice day!
Public speaking is very easy.
Phase 1 defused. How about the next one?
1 2 6 24 120 720
That's number 2. Keep going!
1 b 214
Halfway there!
9
So you got that one. Try this one.
opekmq
Good work! On to the next...
4 2 6 3 1 5
Congratulations! You've defused the bomb!
(james㉿kali)-[~/Labs/Lab1]
$
"bomb" | 26.3 KiB (26943 bytes) | Executable